

Algoritmen en complexiteit

1001WETALC

Doel: Kwantificeer de **computationale complexiteit** van een probleem.

Meer bepaald: We willen weten en meten hoeveel **computationale middelen nodig zijn** om een probleem op te lossen.

- ▶ Tijd
- ▶ Geheugen (Plaats)
- ▶ Randomness
- ▶ ...

En dit op een manier die **intrinsiek** is aan het beschouwde probleem. Met andere woorden, onafhankelijk van

- ▶ Door welke persoon of op welke manier het wordt geïmplementeerd
- ▶ Programmeertaal
- ▶ Gebruikte hardware
- ▶ ...

Doel: Kwantificeer de **computationalele complexiteit** van een probleem.

Meer bepaald: We willen weten en meten hoeveel **computationalele middelen nodig zijn** om een probleem op te lossen.

- ▶ Tijd
- ▶ Geheugen (Plaats)
- ▶ Randomness
- ▶ ...

En dit op een manier die **intrinsiek** is aan het beschouwde probleem. Met andere woorden, onafhankelijk van

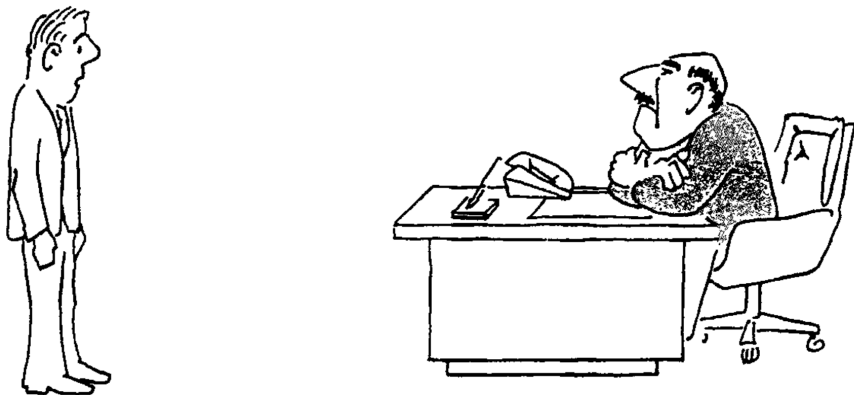
- ▶ Door welke persoon of op welke manier het wordt geïmplementeerd
- ▶ Programmeertaal
- ▶ Gebruikte hardware
- ▶ ...

Wat wil het zeggen als men een probleem als **moeilijk** beschouwt?

Veronderstel dat je baas je een probleem geeft om te implementeren.

Je hebt echter veel moeite om dit efficiënt op te lossen.

Wat doe je in zo'n geval?

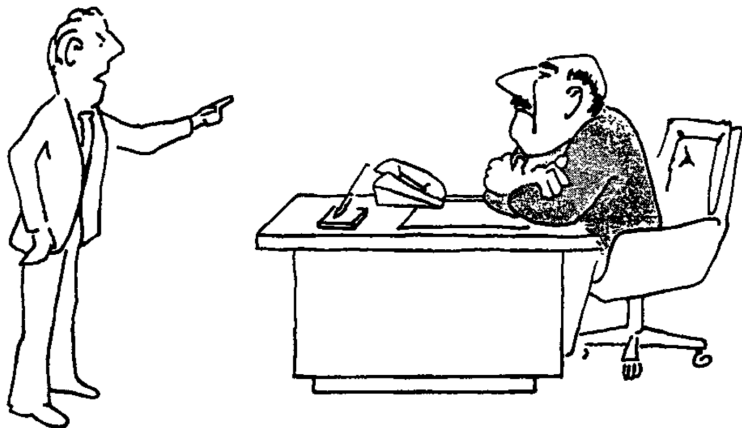


“I can’t find an **efficient** algorithm, I guess I’m just too dumb.”

Misschien was dat antwoord niet zo goed en word je ontslagen...

Dus, in deze cursus leren we methoden om dergelijke vraag beter te beantwoorden!

In het algemeen zijn er twee alternatieve manieren om dit aan te pakken.



“I can’t find an efficient algorithm, because no such algorithm is possible!”



“I can’t find an efficient algorithm, but neither can all these famous people.”

1. Ter herinnering: Turing machines
2. Tijdscomplexiteit
3. Plaatscomplexiteit
4. Relaties tussen verschillende complexiteitsklassen
5. ...

Slides op blackboard (PDF formaat, een printversie en een schermversie, ook beschikbaar op de cursusdienst)

Bronnen:

Michael Sipser. *Introduction to the Theory of Computation*. Second Edition. Thompson Course Technology, 2006.

- ▶ Referenties naar de verwante hoofdstukken staan in de slides.

Sanjeev Arora and Boaz Barak. *Computational Complexity. A modern approach*. Cambridge University Press, 2009.

Christos Papadimitriou. *Computational Complexity*. Addison Wesley, 1994.

Dexter C. Kozen. *Theory of Computation*. Second Edition. Springer-Verlag, 2006.

Doelstellingen:

- ▶ Op een formele manier de moeilijkheidsgraad van een probleem kunnen aanpakken.
- ▶ Verschil tussen efficiënt en niet efficiënt zijn begrijpen en aantonen hoe het hiermee zit met een opgegeven probleem
- ▶ Reducties kunnen maken tussen problemen en de juistheid hiervan kunnen aantonen.
- ▶ Bewijzen begrijpen van belangrijke stellingen in complexiteitstheorie.

Wacht niet tot op het einde met studeren en kom naar de oefeningen op donderdag

Doelstellingen:

- ▶ Op een formele manier de moeilijkheidsgraad van een probleem kunnen aanpakken.
- ▶ Verschil tussen efficiënt en niet efficiënt zijn begrijpen en aantonen hoe het hiermee zit met een opgegeven probleem
- ▶ Reducties kunnen maken tussen problemen en de juistheid hiervan kunnen aantonen.
- ▶ Bewijzen begrijpen van belangrijke stellingen in complexiteitstheorie.

Wacht niet tot op het einde met studeren en kom naar de oefeningen op donderdag

Doelstellingen:

- ▶ Op een formele manier de moeilijkheidsgraad van een probleem kunnen aanpakken.
- ▶ Verschil tussen efficiënt en niet efficiënt zijn begrijpen en aantonen hoe het hiermee zit met een opgegeven probleem
- ▶ Reducties kunnen maken tussen problemen en de juistheid hiervan kunnen aantonen.
- ▶ Bewijzen begrijpen van belangrijke stellingen in complexiteitstheorie.

Wacht niet tot op het einde met studeren en kom naar de oefeningen op donderdag

Doelstellingen:

- ▶ Op een formele manier de moeilijkheidsgraad van een probleem kunnen aanpakken.
- ▶ Verschil tussen efficiënt en niet efficiënt zijn begrijpen en aantonen hoe het hiermee zit met een opgegeven probleem
- ▶ Reducties kunnen maken tussen problemen en de juistheid hiervan kunnen aantonen.
- ▶ Bewijzen begrijpen van belangrijke stellingen in complexiteitstheorie.

Wacht niet tot op het einde met studeren en kom naar de oefeningen op donderdag

Doelstellingen:

- ▶ Op een formele manier de moeilijkheidsgraad van een probleem kunnen aanpakken.
- ▶ Verschil tussen efficiënt en niet efficiënt zijn begrijpen en aantonen hoe het hiermee zit met een opgegeven probleem
- ▶ Reducties kunnen maken tussen problemen en de juistheid hiervan kunnen aantonen.
- ▶ Bewijzen begrijpen van belangrijke stellingen in complexiteitstheorie.

Wacht niet tot op het einde met studeren en kom naar de oefeningen op donderdag

Doelstellingen:

- ▶ Op een formele manier de moeilijkheidsgraad van een probleem kunnen aanpakken.
- ▶ Verschil tussen efficiënt en niet efficiënt zijn begrijpen en aantonen hoe het hiermee zit met een opgegeven probleem
- ▶ Reducties kunnen maken tussen problemen en de juistheid hiervan kunnen aantonen.
- ▶ Bewijzen begrijpen van belangrijke stellingen in complexiteitstheorie.

Wacht niet tot op het einde met studeren en kom naar de oefeningen op donderdag (en naar de les natuurlijk).

Evaluatie:

- ▶ Gesloten boek examen.
- ▶ Examen bestaat uit vragen uit theorie en oefeningenlessen.

Foutjes in cursus? Laat me ze weten: https://docs.google.com/document/d/1emVx18vzn1tMU1aigt84gWEy3TXbIbk_c3P4/edit